

The Job Sequencing and Tool Switching Problem

Part I: Foundations — Formulations, KTNS, and LP Analysis

Scope. This document covers the established theoretical foundations of the Uniform Job Sequencing and Tool Switching Problem (U-SSP): the formal problem definition, the Tang–Denardo decomposition, the KTNS optimal tooling policy, and a rigorous analysis of why classical compact ILP formulations fail computationally. All results in this document are from the literature and are attributed accordingly.

Contents

1	Problem Definition: The Uniform SSP	2
1.1	Formal Setup	2
2	Maximal Configurations and Dummy Tool Padding	2
3	The Tang–Denardo Decomposition	3
4	Interval Matrices and the KTNS Policy	3
4.1	0-Blocks and the Interval Property	3
4.2	The Keep Tool Needed Soonest (KTNS) Algorithm	4
4.3	Optimality of KTNS	4
4.4	Illustrated Example	5
5	Compact ILP Formulations	5
5.1	Tang–Denardo Position-Based Model [Tang and Denardo(1988)]	6
5.2	Laporte–Salazar–Semet Arc-Flow Model [Laporte et al.(2004)Laporte, Salazar-González, and Semet]	
5.3	Catanzaro–Gouveia–Labbé Linear Ordering Model [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé]	
5.4	Multi-Commodity Flow Model (SSPMF)	7
6	Why Compact Models Fail: LP Collapse and Symmetry	7
6.1	LP Lower Bound Collapse to Zero	7
6.2	Exponential Symmetry from 0-Block Choices	8
6.3	Combined Implication	8

Problem Definition: The Uniform SSP

A Flexible Manufacturing System (FMS) consists of a CNC machine with an automated tool changer and a magazine of limited capacity b . The machine must process a set of n jobs sequentially, where each job requires a specific subset of tools to be simultaneously loaded in the magazine. Since the total number of distinct tools m typically exceeds b , tools must be swapped in and out as the machine moves from job to job. Each individual tool swap incurs a time penalty. The *Uniform SSP* asks: in what order should the jobs be processed, and which tools should be in the magazine at each step, so that the total number of tool switches is minimised?

Formal Setup

Let $J = \{1, 2, \dots, n\}$ be the set of jobs and $T = \{1, 2, \dots, m\}$ the set of all available tools. The machine has magazine capacity $b \in \mathbb{Z}^+$ with $b < m$. Each job $j \in J$ requires the tool subset $T_j \subseteq T$, and we assume $|T_j| \leq b$ for all j (otherwise the instance is infeasible).

The *job-tool incidence matrix* $\mathbf{A} \in \{0, 1\}^{m \times n}$ encodes these requirements:

$$a_{tj} = \begin{cases} 1 & \text{if } t \in T_j, \\ 0 & \text{otherwise.} \end{cases}$$

Definition 1.1 (Feasible magazine sequence). A sequence of magazine states $\mathcal{M} = (M_1, M_2, \dots, M_n)$ is *feasible* for job permutation $\sigma \in \Sigma_n$ if, for every step k :

$$|M_k| \leq b \quad \text{and} \quad T_{\sigma(k)} \subseteq M_k.$$

The cost of transitioning from state M_{k-1} to M_k is the number of tool insertions required:

$$\text{cost}(M_{k-1} \rightarrow M_k) = |M_k \setminus M_{k-1}|.$$

Definition 1.2 (U-SSP objective). Find a permutation $\sigma^* \in \Sigma_n$ and a feasible magazine sequence $\mathcal{M}^*$ minimising

$$Z_{\text{SSP}}^* = \min_{\sigma, \mathcal{M}} \sum_{k=1}^{n-1} |M_{k+1} \setminus M_k|.$$

Remark 1.3. The U-SSP is **strongly NP-hard** [Crama et al.(1994)Crama, Kolen, Oerlemans, and Spieksma]: it subsumes the Hamiltonian cycle problem. Even the tooling subproblem (optimal tool loading for a *fixed* sequence) is polynomial [Tang and Denardo(1988)], as shown below.

Maximal Configurations and Dummy Tool Padding

In practice a job may require fewer tools than b , leaving slack slots. A key structural lemma says we can always saturate those slots with *dummy* tools without increasing cost.

Lemma 2.1 (Optimality of maximal configurations [Tang and Denardo(1988)]). *For any feasible magazine sequence $\mathcal{M}$, there exists a sequence of maximal configurations $\mathcal{C} = (C_1, \dots, C_n)$ with $|C_k| = b$ and $T_{\sigma(k)} \subseteq C_k$ for all k , whose total switching cost is $\leq$ that of $\mathcal{M}$.*

Proof. It suffices to show that padding a single under-capacity state M_k with $|M_k| < b$ by one tool never increases cost. We identify the best tool to add by considering two cases.

Case 1: retain a tool from the previous state. Pick any $t \in M_{k-1} \setminus M_k$ and set $M'_k = M_k \cup \{t\}$. The inflow cost $|M'_k \setminus M_{k-1}| = |M_k \setminus M_{k-1}|$ is unchanged (because t was already in M_{k-1}). The outflow cost $|M_{k+1} \setminus M'_k| \leq |M_{k+1} \setminus M_k|$ (since $M'_k \supset M_k$). Net change ≤ 0 .

Case 2: pre-load a future tool. If $M_{k-1} \subseteq M_k$, pick $t \in M_{k+1} \setminus M_k$ and set $M'_k = M_k \cup \{t\}$. Inflow cost increases by +1 (since $t \notin M_{k-1}$). Outflow cost decreases by -1 (since $t \in M_{k+1}$). Net change = 0.

Case 3: arbitrary padding. If both adjacent sets are already subsets of M_k , any dummy tool can be added; the net change is also 0.

Applying this padding iteratively to every under-capacity state yields a maximal sequence with cost $\leq$ the original. $\square$ $\square$

Consequence. The search space can be restricted to maximal configurations of size exactly b without loss of generality.

The Tang–Denardo Decomposition

A central insight of [Tang and Denardo(1988)] is that the U-SSP decomposes into two nested subproblems.

Definition 3.1 (Sequencing Problem (SP)). Given job set J , find the permutation $\sigma^* \in \Sigma_n$ that minimises the cost of the inner Tooling Problem.

Definition 3.2 (Tooling Problem (TP)). Given a fixed permutation σ , find the sequence of maximal magazine configurations $(C_1, \dots, C_n)$ with $T_{\sigma(k)} \subseteq C_k$ and $|C_k| = b$ that minimises

$$TP(\sigma) = \min_C \sum_{k=1}^{n-1} |C_{k+1} \setminus C_k|.$$

The global objective is therefore $Z_{\text{SSP}}^* = \min_{\sigma} TP(\sigma)$. The decomposition reveals two critical structural facts:

1. The TP is **solvable in polynomial time** for any fixed σ (via the KTNS algorithm described next).
2. The SP **cannot** be solved by static TSP-style edge weights, because the transition cost from job i to job j depends on the full history of the magazine state leading up to that transition.

Proposition 3.3 (Failure of static greedy sequencing [Tang and Denardo(1988)]). *Any algorithm that sequences jobs using fixed, precomputed edge costs $c_{ij} = \max\{0, |T_i \cup T_j| - b\}$ cannot guarantee global optimality for the U-SSP.*

Proof. The cost c_{ij} is the *minimum* number of switches required to process job j immediately after job i in a two-job subsequence, ignoring the magazine contents inherited from earlier jobs. In reality, the optimal tooling policy for a long prefix fills the slack slots $C_k \setminus T_{\sigma(k)}$ with tools needed by future jobs. A greedy algorithm evaluating only c_{ij} is blind to these pre-loaded tools. It may select a successor job j' that appears cheaper under $c_{ij'}$ but causes subsequent transitions to be far more expensive because the look-ahead pre-load is destroyed. The true cost is thus a function of the entire prefix permutation, not a pairwise quantity. $\square$ $\square$

Interval Matrices and the KTNS Policy

0-Blocks and the Interval Property

Fix a permutation σ and consider the permuted incidence matrix $\mathbf{A}^\sigma \in \{0, 1\}^{m \times n}$ whose columns are ordered by σ .

Definition 4.1 (0-block). A 0-block for tool t is a maximal contiguous run of zeros in row t of $\mathbf{A}^\sigma$ that is bounded on both sides by a 1. The interval $[k_1, k_2]$ is a 0-block if $a_{t,k}^\sigma = 0$ for $k \in [k_1, k_2]$, while $a_{t,k_1-1}^\sigma = 1$ and $a_{t,k_2+1}^\sigma = 1$.

A 0-block represents an idle window during which tool t is not required, yet will be needed again later. The decision to *retain* t across this gap (“flipping” the 0-block) avoids a future re-insertion but occupies a magazine slot in the meantime. If every 0-block of every tool is flipped, the matrix has the *Consecutive Ones Property (COP)*: all 1s in each row form a single contiguous block. Such matrices are called *interval matrices* and are **totally unimodular**, so the TP LP relaxation has integer optimal solutions [Crama et al.(1994)Crama, Kolen, Oerlemans, and Spieksma].

The Keep Tool Needed Soonest (KTNS) Algorithm

The KTNS policy resolves the TP optimally: whenever a slot must be freed to load a required tool, evict the resident tool whose *next* required use is *furthest in the future*. Let

$$L(t, k) = \min\{s \geq k : t \in T_{\sigma(s)}\}$$

denote the next requirement step for tool t at or after step k (set to $n + 1$ if t is never needed again).

Algorithm 1: Keep Tool Needed Soonest (KTNS)

Input: Permutation $\sigma \in \Sigma_n$, requirement sets $T_{\sigma(k)}$, capacity b .

Output: Optimal magazine sequence $(C_1, \dots, C_n)$ and cost Z_{TP} .

1. Precompute $L(t, k) = \min\{s \geq k : t \in T_{\sigma(s)}\}$ for all $t \in T$, $k \in \{1, \dots, n\}$. Set $L(t, k) = n + 1$ if t is never needed from k onward.
2. Set $C_1 \leftarrow T_{\sigma(1)}$; pad to size b by adding tools with the smallest values of $L(\cdot, 1)$. Set $Z_{TP} \leftarrow 0$.
3. **For** $k = 2, 3, \dots, n$:
 - a. Set $C_k \leftarrow C_{k-1}$ (inherit the previous magazine state).
 - b. **For each** required tool $t \in T_{\sigma(k)} \setminus C_k$:
 - i. Choose the **KTNS eviction candidate**: $p \leftarrow \arg \max_{s \in C_k \setminus T_{\sigma(k)}} L(s, k)$ (evict the tool needed furthest in the future).
 - ii. Update: $C_k \leftarrow (C_k \setminus \{p\}) \cup \{t\}$. Increment: $Z_{TP} \leftarrow Z_{TP} + 1$.
4. **Return** $(C_1, C_2, \dots, C_n)$ and Z_{TP} .

Time complexity: $O(n \cdot m \cdot b)$.

Remark 4.2. KTNS runs in $O(n \cdot m \cdot b)$ time and is analogous to Belady’s optimal page-replacement algorithm [Belady(1966)].

Optimality of KTNS

Theorem 4.3 (KTNS is optimal for the TP [Tang and Denardo(1988)]). *For any fixed permutation σ and capacity b , KTNS produces a magazine sequence achieving the minimum switching cost $TP(\sigma)$.*

Proof. We use an exchange argument. Let π be any feasible tooling policy for σ . We show how to transform π step-by-step into KTNS without ever increasing cost.

Setup. Let k be the first step at which π disagrees with KTNS. At step k , a required tool $r \in T_{\sigma(k)} \setminus W_k$ must be inserted (where W_k is the shared magazine state just before the eviction), so one tool from $W_k \setminus T_{\sigma(k)}$ must be evicted. KTNS evicts $t^* = \arg \max_{s \in W_k \setminus T_{\sigma(k)}} L(s, k)$, but π evicts a different tool s with $L(s, k) < L(t^*, k)$.

Construction of π' . Define π' identical to π except that at step k it evicts t^* (agreeing with KTNS). Let $p = L(s, k)$ and $q = L(t^*, k)$, so $p \leq q$.

Cost comparison during $[k, q]$.

- **Step k :** both π and π' insert r (one insertion each); the only difference is who else occupies the freed slot.
- **Steps $k+1$ to $p-1$:** neither s nor t^* is required (since p and q are their next uses). Both policies are feasible and handle all other required tools identically; cost is the same.
- **Step p** (s first needed again): π already has s — no extra insertion. π' does not have s — must insert s (+1 insertion for π'); π' can evict t^* (still unneeded until step q) to free the slot.
- **Steps $p+1$ to $q-1$:** t^* not needed yet. π retains t^* ; π' does not have t^* . Both handle all other tools identically.
- **Step q** (t^* first needed again): If π still has t^* : π pays 0 for t^* ; π' must insert t^* (+1 insertion for π'). Net over $[k, q]$: π' paid +1 at step p and +1 at step q , while π paid +1 at step p (for inserting s) and 0 at step q . Total: $\pi' = 2$ extra, $\pi = 1$ extra — *but we overcounted*.

Correcting the count. The insertion of r at step k is paid by *both* policies equally and does not enter the comparison. The comparison reduces to:

Event	Policy π	Policy π'
Insert s at step p	+1	0
Insert t^* at step q	0	+1

The totals are equal: both policies pay exactly **1 insertion** for the pair $\{s, t^*\}$ over the interval $[k, q]$.

If instead π evicts t^* before step q (say at step p when inserting s), then π also pays 1 for t^* at step q — again equal to π' . In all cases, $\text{cost}(\pi') \leq \text{cost}(\pi)$.

Cascading differences. At step p , π' evicts t^* while π may evict a different tool u . This creates a new disagreement (in u) between π and π' . The same exchange argument applies to this new disagreement: evicting the further-needed tool never increases cost. Since the sequence has finite length n , applying the exchange argument inductively reduces the number of disagreements to zero, transforming π into KTNS without increasing the total cost. Hence $\text{cost}(\text{KTNS}) \leq \text{cost}(\pi)$ for any feasible π , proving KTNS is optimal. $\square$ $\square$

Illustrated Example

Compact ILP Formulations

Early exact approaches modelled the SSP as a single Integer Linear Program over positional or arc-flow variables. We survey the four main formulations.

	Job 1	Job 2	Job 3	Job 4	Job 5
t_1	Required	Retained	Retained	Required	Required
t_2	Required	Required	Evicted	—	Required
t_3	Required	Required	Evicted	—	—
t_4	Padded	Required	Required	Evicted	—
t_5	—	—	Required	Required	Retained
t_6	—	—	Required	Required	Evicted
t_7	—	—	—	—	Required

Required by job
 KTNS: 0-block
 Not in magazine

Figure 1: KTNS magazine evolution for a 5-job, 7-tool instance ($b = 4$). Tool t_1 is retained across jobs 2–3 (0-block flipped) because capacity allows it. Tool t_2 's 0-block cannot be flipped at job 3 because $T_3 = \{t_4, t_5, t_6\}$ plus the retained t_1 already saturates all 4 slots; t_2 must be evicted and re-inserted before job 5.

Tang–Denardo Position-Based Model [Tang and Denardo(1988)]

Variables.

- $u_{jk} \in \{0, 1\}$: 1 if job j is assigned to position k .
- $v_{kt} \in \{0, 1\}$: 1 if tool t is in the magazine at position k .
- $w_{kt} \geq 0$: number of insertions of tool t at position k .

$$\min \sum_{k,t} w_{kt} \quad (1)$$

$$\text{s.t.} \quad \sum_k u_{jk} = 1 \quad \forall j, \quad \sum_j u_{jk} = 1 \quad \forall k \quad (2)$$

$$v_{kt} \geq \sum_j a_{tj} u_{jk} \quad \forall k, t \quad (3)$$

$$\sum_t v_{kt} \leq b \quad \forall k \quad (4)$$

$$w_{kt} \geq v_{kt} - v_{k-1,t} \quad \forall k, t \quad (5)$$

$$u_{jk}, v_{kt} \in \{0, 1\}, w_{kt} \geq 0$$

Boundary condition: $v_{0,t} = 0$ for all t .

Laporte–Salazar–Semet Arc-Flow Model [Laporte et al.(2004)Laporte, Salazar-González, and Semet]

This model embeds the SP in an ATSP framework over jobs $J_0 = J \cup \{0\}$ (with dummy depot 0). Variables: $x_{ij} \in \{0, 1\}$ (routing arc), $y_{tj} \in \{0, 1\}$ (tool t present at job j), $z_{tj} \in \{0, 1\}$ (tool t

inserted at job j).

$$\begin{aligned} \min \quad & \sum_{j,t} z_{tj} \\ \text{s.t.} \quad & \sum_{i \neq j} x_{ij} = \sum_{i \neq j} x_{ji} = 1 \quad \forall j \end{aligned} \quad (6)$$

Subtour elimination constraints (SECs)

$$y_{tj} \geq a_{tj} \quad \forall j, t \quad (7)$$

$$\sum_t y_{tj} \leq b \quad \forall j \quad (8)$$

$$z_{tj} \geq y_{tj} - y_{ti} + x_{ij} - 1 \quad \forall i \neq j, \forall t \quad (9)$$

Constraint (9) avoids Big-M: if $x_{ij} = 1$ (job j directly follows i) it forces $z_{tj} \geq y_{tj} - y_{ti}$, i.e., tool t must be inserted at j if it was absent at i . When $x_{ij} = 0$ the constraint is inactive.

Catanzaro–Gouveia–Labbé Linear Ordering Model [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé]

Instead of absolute positions, binary variable $x_{ij} = 1$ encodes that job i is processed *before* j (not necessarily immediately). Tournament and transitivity constraints enforce a valid linear order:

$$x_{ij} + x_{ji} = 1 \quad \forall i \neq j \quad (10)$$

$$x_{ij} + x_{jk} + x_{ki} \leq 2 \quad \forall i \neq j \neq k \quad (11)$$

The objective maximises retained 0-blocks directly. A retention variable $y_{ij}^t = 1$ indicates that tool t (required by both i and j) is kept continuously between i and j without any intermediate job needing it. Overlap capacity constraints prevent the simultaneous retention of too many tools at any intermediate job k :

$$\sum_{t \notin T_k} \sum_{i: t \in T_i} \sum_{j: t \in T_j} y_{ij}^t (x_{ik} + x_{kj} - 1) \leq b - |T_k| \quad \forall k \in J.$$

This model achieves the tightest LP relaxation among compact formulations [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé].

Multi-Commodity Flow Model (SSPMF)

A further tightening treats tool continuity as a network flow. For each tool t , flow variable $f_{ij}^t \in \{0, 1\}$ indicates that t is continuously retained across the direct arc $i \rightarrow j$. Flow conservation ties retention to insertion cost:

$$z_{tj} \geq y_{tj} - \sum_{i \neq j} f_{ij}^t \quad \forall j, t.$$

Arc capacity: $\sum_t f_{ij}^t \leq b \cdot x_{ij}$. This formulation explicitly tracks tool lifespans across the sequence, providing tighter LP bounds than the basic arc-flow model, but still cannot fully escape the symmetry problem described next [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé].

Why Compact Models Fail: LP Collapse and Symmetry

All four compact models above suffer from two interrelated weaknesses that make them computationally intractable beyond small instances.

LP Lower Bound Collapse to Zero

Theorem 6.1 (LP bound collapse [Tang and Denardo(1988), Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé]). *The LP relaxation of the Tang–Denardo position-based model admits a feasible solution with objective value 0, regardless of the true integer optimum.*

Proof. Set $u_{jk} = 1/n$ for all j, k ; this satisfies the doubly-stochastic assignment constraints (2). The coverage constraint (3) then forces

$$v_{kt} \geq \frac{1}{n} \sum_j a_{tj} \quad \forall k, t.$$

Set $v_{kt} = \frac{1}{n} \sum_j a_{tj}$ (independent of k). The capacity constraint (4) is satisfied because $\sum_t v_{kt} = \frac{1}{n} \sum_j |T_j| \leq b$. Since v_{kt} is constant in k , the gradient $v_{kt} - v_{k-1,t} = 0$ everywhere, so the transition variables $w_{kt} = 0$ are feasible. The objective evaluates to $\sum_{k,t} w_{kt} = 0$. $\square$ $\square$

Remark 6.2. The same collapse occurs in the arc-flow and multi-commodity models: the LP solver can “smear” job assignments uniformly across all arcs, making every tool appear at every node with fractional presence, eliminating all apparent transitions.

Exponential Symmetry from 0-Block Choices

Even when variables are fixed to $\{0, 1\}$, compact models suffer from *permutational symmetry*: the binary choice of whether to retain or evict a tool during each 0-block generates exponentially many equivalent integer solutions.

Theorem 6.3 (2^k 0-block symmetry [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé]). *If a fixed sequence contains k independent 0-blocks (whose simultaneous retention does not violate capacity), then the compact model has at least 2^k feasible integer variable assignments that all incur the same objective cost, creating a flat plateau of size 2^k in the branch-and-bound tree.*

Sketch. Each independent 0-block for tool t_i spanning interval $[p_i, q_i]$ presents a binary choice $\delta_i \in \{0, 1\}$: retain ($\delta_i = 1$) or evict-and-reload ($\delta_i = 0$). By independence (capacity is not tight), each choice δ_i is feasible regardless of the others. Since the intervals are disjoint by independence, the objective cost is the same for all 2^k combinations of $\{\delta_i\}$: evicting costs one insertion (to re-load at $q_i + 1$), but so does entering the 0-block having not loaded the tool previously (one insertion at the first required position). The branch-and-bound must distinguish these 2^k equivalent configurations before proving optimality, causing exponential stalling. $\square$ $\square$

Combined Implication

The LP collapse to 0 means the root-node bound gives no useful pruning information. The 2^k symmetry means the search tree branches on combinatorially equivalent choices at every node. Together, these render all four compact formulations impractical for instances with more than roughly 25 jobs [Laporte et al.(2004)Laporte, Salazar-González, and Semet, Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé]. This motivates the *non-compact, configuration-based formulation* developed in the companion document `02_noncompact_formulation.tex`.

References

- [Belady(1966)] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé] D. Catanzaro, L. Gouveia, and M. Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.
- [Crama et al.(1994)Crama, Kolen, Oerlemans, and Spieksma] Y. Crama, A. W. J. Kolen, A. G. Oerlemans, and F. C. R. Spieksma. Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6(1):33–54, 1994.
- [Laporte et al.(2004)Laporte, Salazar-González, and Semet] G. Laporte, J. J. Salazar-González, and F. Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.
- [Tang and Denardo(1988)] C. S. Tang and E. V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.